

Secure Chat Application with RSA Encryption

Team Members:

1. *Himavarshini.Kolisetty(24228)*
2. *Poojitha.Devineni(24263)*
3. *Amrutha.Tammuroth(24255)*
4. *Rupesh.Namburi(24234)*
5. *Karthik.gandrothu(24217)*
6. *Uday Sri.Yaramati(24260)*



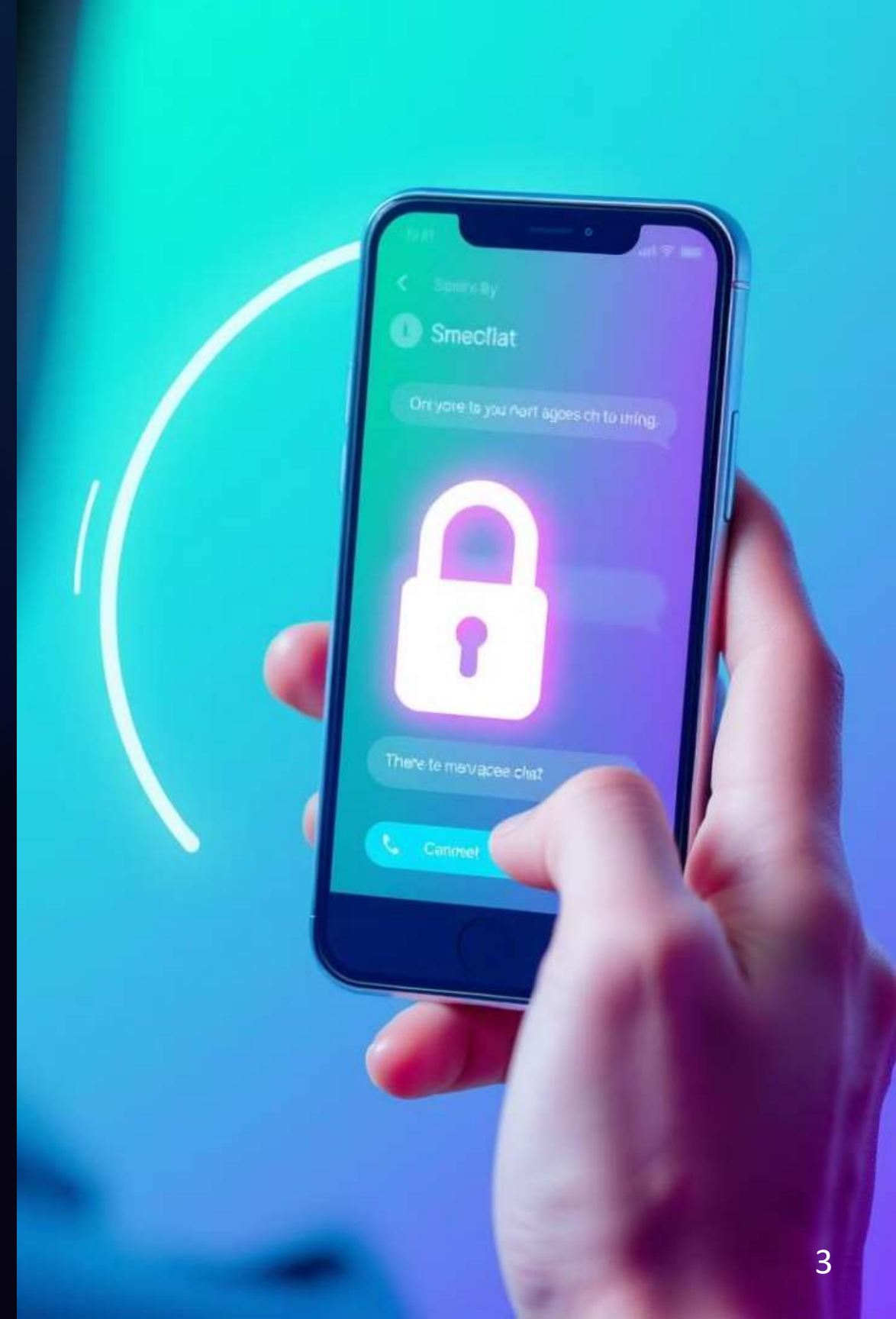
RSA
ENCRYPTION ARDO

Contents

- 1 Introduction
- 2 Literature Review
- 3 Research Gaps
- 4 Problem Statement
- 5 Objectives
- 6 Methodology
- 7 Results
- 8 Conclusion and Future Scope

Introduction

RSA encryption, developed by Rivest, Shamir, and Adleman in 1978, is a public-key cryptosystem that uses large prime numbers to generate a secure pair of keys for encrypting and decrypting data. RSA is essential for secure data exchange across open networks, where privacy and data integrity are paramount. By leveraging the computational complexity of prime factorization, RSA ensures that only intended recipients can decrypt messages. Our project implements RSA encryption in a chat application, where user messages are encrypted before being transmitted, ensuring confidentiality and preventing unauthorized access.



Literature Review

Author/Title/Year	Methodologies and Techniques	Key Contributions
Diffie, W., & Hellman, M. (1976)	Introduced public-key cryptography	Laid the groundwork for RSA's two-key model by proposing the key-pair generation concept for secure sharing.
Rivest, Shamir & Adleman (1978)	Developed RSA using modular exponentiation	Defined RSA, now fundamental to secure data exchange.
Boneh, D., & Franklin, M. (2001)	Identity-Based Encryption (IBE) in RSA	Made RSA suitable for identity-based encryption, simplifying secure communications in web apps.
Bellare, M., & Rogaway, P. (1995)	Probabilistic Signature Scheme (PSS) for RSA	Enhanced RSA's security for digital signatures, ensuring message integrity.
Koblitz, N. (1994)	Proposed ECC as an alternative	Highlighted computational efficiencies of ECC over RSA, noting RSA's resource demands.
Lenstra, A. K., & Verheul, E. R. (2001)	Key size and security strength analysis for RSA	Recommended larger RSA key lengths to match advances in computational power.
Shamir, A. (1997)	Timing attack vulnerability on RSA	Showed that RSA could be susceptible to timing attacks, leading to more secure implementations.
Hinek, M. J., Low, M. K., & Teske, E. (2008)	Efficient RSA key distribution	Improved RSA-based key distribution, key for applications like secure multicast.
Wang, X., & Yu, H. (2005)	Cryptanalysis of RS/  signatures	Identified weaknesses in certain RSA signature schemes, pushing for improved methods.

Research Gaps

Scalability Issues

RSA encryption can struggle
in real-time applications
with high message volumes
and user counts.

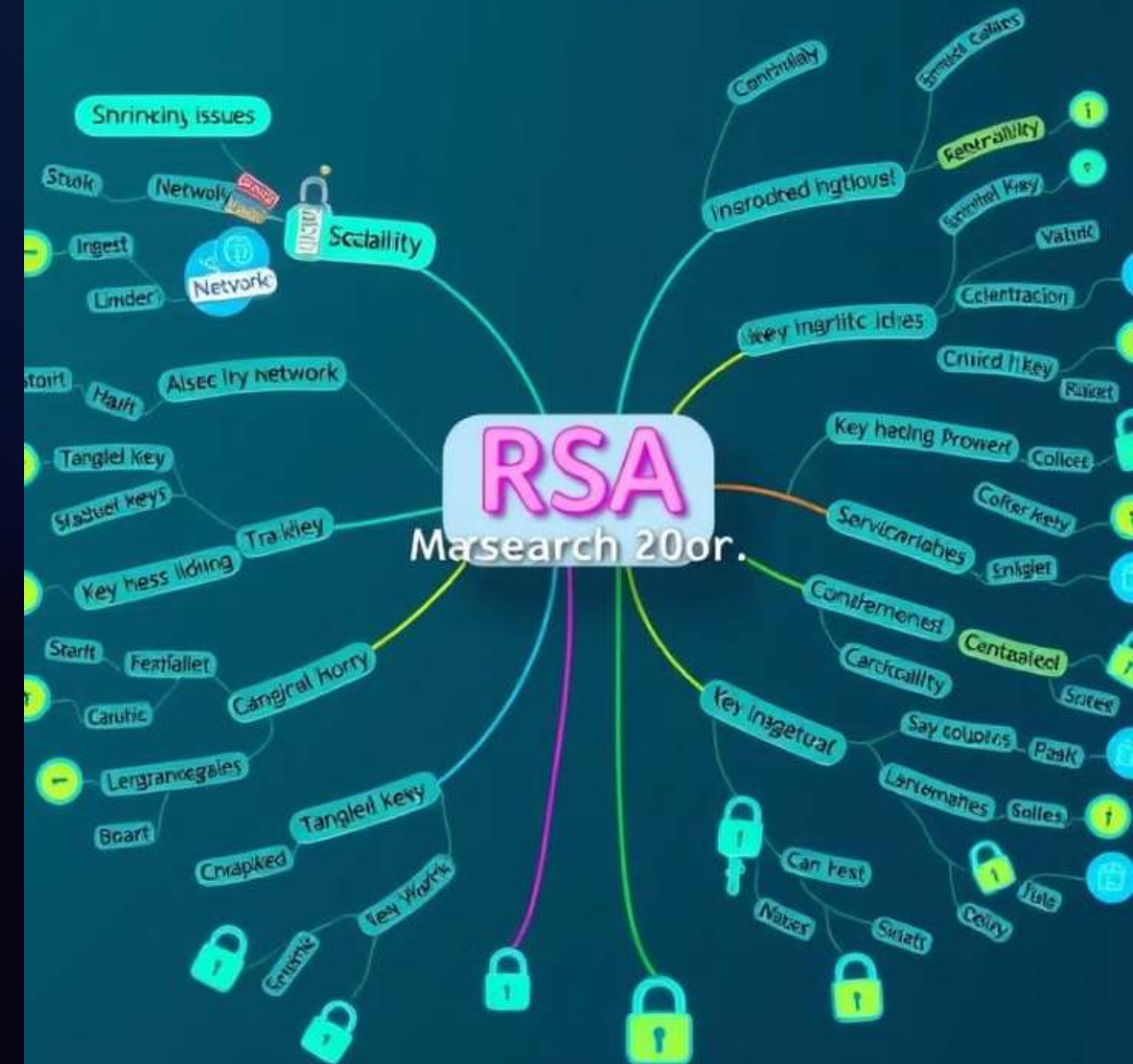
Key Management Complexities

Efficiently generating,
distributing, and storing RSA
keys remains challenging.

Potential Vulnerabilities

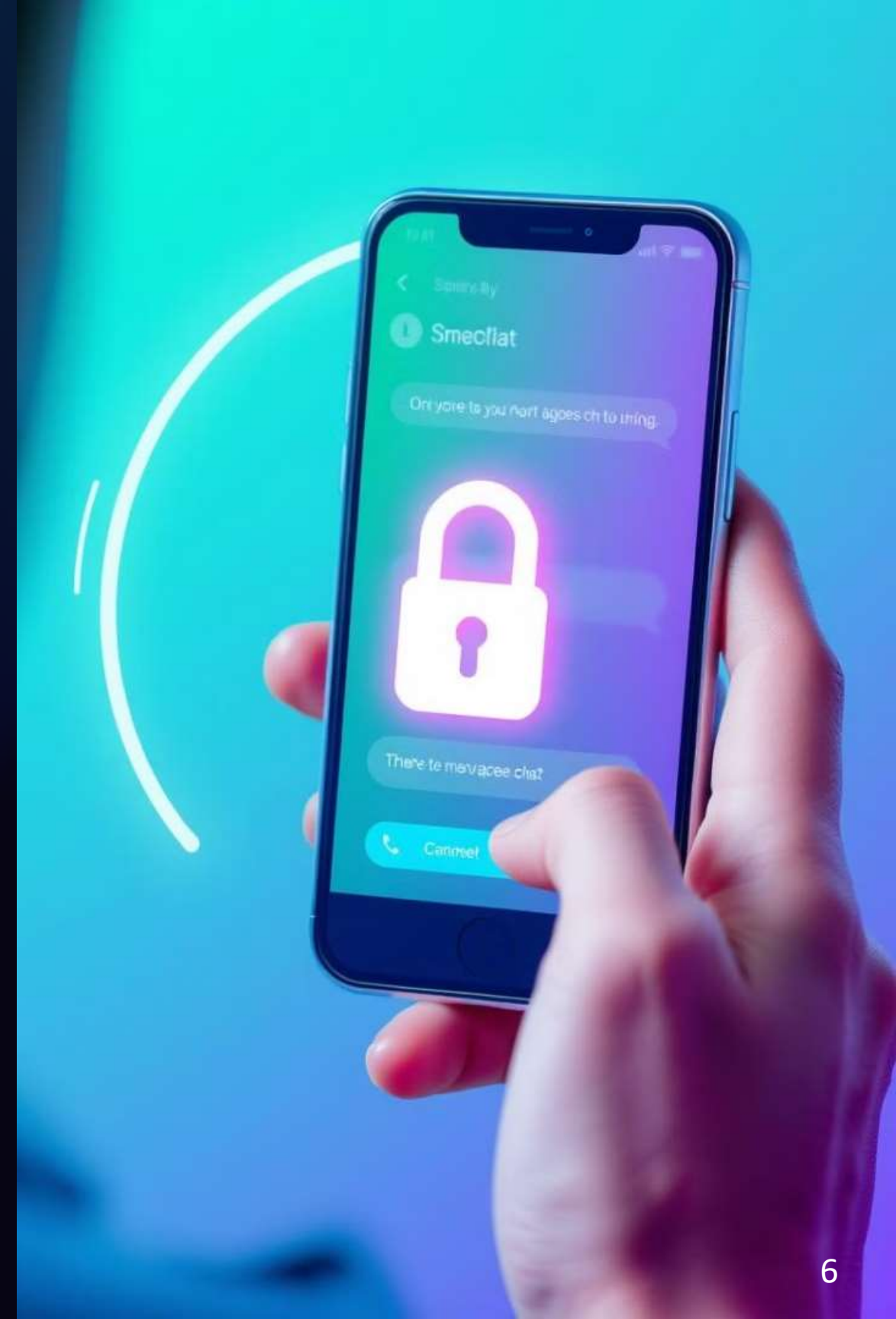
RSA's susceptibility to timing attacks and reliance on large key sizes highlights the need for additional security measures.

Key Research Gaps



Problem Statement

Our project addresses the issue of secure, scalable real-time communication. With increasing privacy concerns, there's a need for applications that provide end-to-end encryption to prevent unauthorized access to sensitive information. Our secure chat application implements RSA encryption, aiming to protect users' messages from being intercepted or read by unintended parties. Additionally, we've incorporated features such as message expiry, encoding and decoding, time-locked messages, and tampering detection to further enhance message security and privacy.





Objectives

1

Develop

A chat application that uses RSA encryption for message security.

2

Implement

A secure and user-friendly key exchange protocol.

3

Ensure

The application can handle multiple users simultaneously without compromising performance.

4

Incorporate

Additional features, including message expiry, encoding and decoding, time-locked messages, and tampering detection.

Methodology

Server.py

Manages client connections, routes encrypted messages, and coordinates the exchange of public keys.

Client.py

Allows users to connect to the server, encrypts outgoing messages, and decrypts incoming messages.

Rsa_keys.py

Handles RSA key pair generation. It includes encoding and decoding functions to optimize message size and speed, improving transmission efficiency.

Results and Discussion



Message Encryption and Decryption

Messages are successfully encrypted on the sender's side and decrypted only by the recipient.



Message Expiry

Messages are deleted from the server after their expiry time, ensuring sensitive information isn't stored beyond its useful period.



Encoding and Decoding

Encoding reduced transmission size, improving performance under high message loads, with decoding preserving data integrity.



Tampering Detection

Attempts to alter encrypted messages were successfully flagged, preserving trust in message content.



Time Locked Messages

Messages with time locks remained encrypted until the specified time, providing controlled access to sensitive information.

Conclusions and Future Scope

1

Add Group Chat Functionality

Expanding RSA encryption to handle multiple recipients simultaneously.

2

Integrate User Authentication

Strengthening identity verification for secure key exchange

3

Enhance Tampering Detection

Applying advanced hashing algorithms to further improve tampering detection accuracy.



References

1. R. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120-126, 1978, doi: 10.1145/359340.359342.
2. W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.
3. M. Naor and B. Pinkas, "Cryptographic principles of secure communication systems," *IEEE Security & Privacy*, vol. 1, no. 2, pp. 33-40, Mar.-Apr. 2003, doi: 10.1109/MSECP.2003.1195609.

t h a n k y o u